

STAJDESK

GÜN 1 → GÜN 12

YAZILIMA GİRİŞTEN OOP'YE

Sıfırdan öğrenmek için hazırlanmış ders kitabı

BU KİTAP NE İÇİN?

Bu doküman bir özet kağıdı değil. Konuları ilk defa öğrenen bir öğrencinin takip edebileceği şekilde hazırlanmıştır. Her bölümde önce günlük hayattan bir benzetme, sonra kavram, ardından kod ve en sonda küçük bir kontrol bulunur.

İÇİNDEKİLER

- Bölüm 1 — Git, Terminal ve Web (Gün 1-3)
- Bölüm 2 — Docker (Gün 4-6)
- Bölüm 3 — C# Temelleri (Gün 7-9)
- Bölüm 4 — OOP ve StajDesk (Gün 10-12)
- Son bölüm — Büyük resim ve tekrar soruları

StajDesk • Lise Stajyer Programı

BU KİTAP NASIL ÇALIŞILIR?

Kodu sadece okumak yerine küçük adımlarla çalıştır.

- 1 — OKU → Kavramın ne olduğunu anla.
- 2 — BENZETMEYİ HATIRLA → Gerçek hayattaki karşılığını kur.
- 3 — KODU YAZ → Kopyalamak yerine kendin tekrar yaz.
- 4 — DEĞİŞTİR → Örnekteki değeri değiştirip sonucu gözlemle.
- 5 — KENDİ CÜMLENLE ANLAT → Mentora anlatabiliyorsan gerçekten öğrenmişsindir.

Bir örnek: “class nedir?”

Sadece “class sınıftır” demek yetmez. Şunu söyleyebilmelisin: “Class, nesnelerin hangi özelliklere ve davranışlara sahip olacağını tanımlayan şablondur. Örneğin Ticket class'ı bir destek talebinin Id, Title ve Status gibi bilgilerini tanımlar.”

KİTABIN ANA FİKRİ

Terminal bilgisayarla konuşmanı sağlar. Git yaptığın değişiklikleri takip eder. HTTP sistemlerin konuşmasını sağlar. Docker ortamı standartlaştırır. C# uygulamanın kodudur. LINQ veriyi sorgular. OOP kodu anlamlı nesnelere böler. Repository veri erişimini ayırır. async/await beklemeli işlemleri daha verimli yönetmeye yardım eder.

BÖLÜM 1

Git, Terminal ve Web'in Çalışma Mantığı

GÜN 1/48 — ORTAM KURULUMU

Hedef: Staj boyunca kullanacağın araçların ne işe yaradığını anlamak ve bilgisayarını hazırlamak.

1. Yazılım geliştirirken neden bu kadar araç var?

Bir marangozun sadece çekiçle çalışmadığını düşün. Ölçü aleti, testere, tornavida ve başka araçlara ihtiyacı vardır. Yazılım geliştiricinin araçları da benzer şekilde farklı görevleri çözer.

VS Code Kodları yazdığın editör.	Git Kod değişikliklerini takip eden sürüm kontrol sistemi.
.NET SDK C#/.NET uygulamalarını oluşturup çalıştırmak için gerekli araçlar.	Node.js JavaScript tabanlı araçları çalıştırmak için kullanılan çalışma ortamı.
Docker Desktop Container'ları bilgisayarında çalıştırmaya yardım eder.	Jira Ekipte yapılacak işlerin takip edilmesini sağlar.

2. Terminal nedir?

Terminal, bilgisayara grafik arayüz yerine komut yazarak işlem yaptığın yerdir. Bir klasöre girmek, dosya oluşturmak veya program çalıştırmak için kullanabilirsin.

```
dotnet --version
node -v
git --version
docker --version
```

Bu komutlar programların kurulu olup olmadığını ve sürümlerini kontrol eder.

3. LOG.md neden var?

LOG.md'yi staj günlüğün gibi düşün. Örneğin bugün "Git'te commit ile push arasındaki farkı öğrendim" yazabilirsin.

Günün ana fikri: Araçları ezberleme. Her aracın hangi problemi çözdüğünü bil.

GÜN 2/48 — TERMİNAL VE GİT

Hedef: Terminalde rahat hareket etmek ve Git'in temel çalışma mantığını kavramak.

1. Dosya yolu mantığı

Bilgisayarındaki dosyaları bir apartmandaki daireler gibi düşün. Klasörler odalar, dosyalar odaların içindeki eşyalar gibidir. Terminalde hangi odada olduğunu bilmek önemlidir.

```
cd proje # klasöre gir
cd .. # üst klasöre çık
dir # Windows'ta içeriği göster
mkdir test # klasör oluştur
```

Mutlak yol tam adresi söyler. Göreli yol bulunduğun yere göre adresi söyler.

2. Git nedir?

Git'i projenin fotoğraf albümü gibi düşünebilirsin. Projede önemli bir noktaya geldiğinde commit alırsın. Sonra bir hata yaparsan geçmişteki noktaya bakabilirsin.

3. Git'in kelimeleri

- Repository (repo): Git'in takip ettiği proje.
- Commit: Projenin belirli bir andaki kayıt noktası.
- Branch: Ana kodu bozmadan ayrı çalışabileceğin dal.
- Merge: Bir branch'teki değişiklikleri başka branch'e birleştirme.
- Remote: İnternette/şirket sunucusunda bulunan uzak repo.

4. Commit ve Push

```
git add .
git commit -m "Login sayfası eklendi"
git push
```

Buradaki akış: değişiklikleri seç → yerel Git geçmişine kaydet → uzak depoya gönder.

Çok önemli: Commit yaptığında GitHub'a göndermiş olmazsın. GitHub'a göndermek için push gerekir.

GÜN 3/48 — WEB NASIL ÇALIŞIR?

Hedef: Tarayıcıya bir adres yazdığında arka planda ne olduğunu temel seviyede açıklayabilmek.

1. Client ve Server

Bir restoran düşün. Sen müşteri (client), mutfak server gibi davranır. Sen sipariş verirsin; mutfak isteği işler ve sonucu geri gönderir. Web'de tarayıcı istek gönderir, sunucu cevap verir.

```
Tarayıcı (Client)
|
| HTTP Request
▼
Server
|
| HTTP Response
▼
Tarayıcı
```

2. URL ve DNS

URL, ulaşmak istediğin kaynağın adresidir. DNS ise alan adını bilgisayarların kullandığı IP adresiyle eşleştirmeye yardımcı olur.

3. HTTP metotları

GET → veri getir
POST → yeni veri oluştur

PUT → mevcut veriyi güncelle
DELETE → veri sil

4. Durum kodları

200 = başarılı. 201 = oluşturuldu. 400 = istek hatalı. 401 = yetkilendirme gerekli. 404 = kaynak bulunamadı. 500 = sunucu tarafında hata.

5. JSON nedir?

JSON, sistemlerin veri alışverişinde sık kullandığı yapılandırılmış bir metin formatıdır.

```
{
  "id": 1,
  "title": "Login problemi",
  "status": "Open"
}
```

Kendini test et: 404 ile 500 arasındaki farkı kendi cümlemlerle anlat. 404 “kaynağı bulamadım”, 500 ise “sunucuda işler ters gitti” gibi düşünülebilir.

BÖLÜM 2

Docker ile Tanışma

GÜN 4/48 — CONTAINER NEDİR?

Hedef: Docker'ın neden kullanıldığını ve image ile container arasındaki farkı anlamak.

1. “Benim bilgisayarımda çalışıyordu” problemi

Bir uygulamanın çalışması sadece kaynak koduna bağlı değildir. İşletim sistemi, kütüphane sürümleri, ayarlar ve başka bağımlılıklar da önemlidir. Bir geliştiricide çalışan uygulama diğerinde çalışmayabilir.

2. Docker'ın fikri

Uygulamayı ihtiyaç duyduğu ortamla birlikte paketleyip farklı bilgisayarlarda daha tutarlı şekilde çalıştırmaya yardım eder.

3. Image ve Container

IMAGE = TARİF / KALIP

Nasıl hazırlanacağını ve gerekli dosyaları içerir.

CONTAINER = O KALIPTAN ÇALIŞAN ÖRNEK

Image'ın çalışan halini düşün.

4. İlk komutlar

```
docker run hello-world
docker run -d -p 8080:80 nginx
docker ps
docker logs CONTAINER_ID
docker stop CONTAINER_ID
docker rm CONTAINER_ID
```

docker ps çalışan container'ları gösterir. logs çıktıları gösterir. stop durdurur. rm container'ı siler.

Benzetme: Image = kek tarifi ve hazırlanmış kalıp. Container = o tariften gerçekten pişirdiğin kek.

GÜN 5/48 — DOCKERFILE

Hedef: Kendi uygulamanın image'ını Dockerfile kullanarak oluşturmanın mantığını öğrenmek.

1. Dockerfile nedir?

Dockerfile, Docker'a "bu image'ı oluştururken şu adımları uygula" diyen talimat dosyasıdır.

2. Komutları anlamlandır

FROM Nereden başla?	COPY Dosyaları içeri al.
WORKDIR Hangi klasörde çalış?	RUN Build sırasında komut çalıştır.
EXPOSE Uygulamanın kullandığı portu belirt.	Katmanlar Docker build adımlarını katmanlar halinde tutar; önbellek build süresini etkileyebilir.
CMD Container başlayınca varsayılan komut.	

3. Örnek Dockerfile

```
FROM nginx
WORKDIR /usr/share/nginx/html
COPY . .
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4. Build ve Run

```
docker build -t stajdesk-site .
docker run -d -p 8080:80 stajdesk-site
```

5. Volume neden önemli?

Container'ın yaşamını bir geçici oda gibi düşün. O oda silinirse içerideki geçici veriler de kaybolabilir. Kalıcı veriyi container'dan ayrı tutmak için volume kullanılır.

Kendini test et: Dockerfile ile image arasındaki fark nedir? Cevap: Dockerfile image'ı nasıl oluşturacağını anlatır; image ise oluşturulmuş kalıptır.

GÜN 6/48 — DOCKER COMPOSE

Hedef: Birden fazla servisi tek proje halinde çalıştırmanın mantığını öğrenmek.

1. Neden birden fazla servis?

StajDesk ilerledikçe sadece tek bir program olmayacak. Örneğin uygulama, PostgreSQL ve Adminer gibi parçalar birlikte çalışabilir.

2. Compose ne yapar?

Compose'u bir orkestratör gibi düşün. "Database'i başlat, Adminer'ı başlat, network oluştur, portları ayarla" gibi işleri tek yapıdan yönetmeye yardım eder.

```
services:
  database:
    image: postgres
    ports:
      - "5432:5432"

  adminer:
    image: adminer
    ports:
      - "8080:8080"
```

3. Network, Environment ve Volume

- Network: Servislerin birbiriyle iletişim kurmasına yardım eder.
- Environment: Şifre, bağlantı adresi gibi ayarları koddan ayırmaya yardım eder.
- Volume: Database gibi kalıcı verilerin container silinse bile korunmasına yardım eder.

4. StajDesk mimarisini gör

```
Kullanıcı
↓
Uygulama / API
↓
Repository
↓
PostgreSQL
```

Günün cümlesi: Compose, birden fazla container'ı birlikte yönetmek için kullanılan yapılandırma aracıdır.

BÖLÜM 3

C# Temelleri

GÜN 7/48 — C# VE .NET

Hedef: C# dilinin temel yapı taşlarını ve ilk .NET console uygulamasını anlamak.

1. .NET neden var?

.NET, C# ile uygulamalar geliştirmek için kullanılan platform ve ekosistemdir. C# güçlü tip sistemi, geniş kütüphaneleri ve kurumsal kullanımıyla sık tercih edilir.

2. İlk proje

```
dotnet new console
dotnet run
```

İlk komut yeni console projesi oluşturur. İkinci komut uygulamayı çalıştırır.

3. Değişkeni kutu gibi düşün

Bir değişken, programın bir bilgiyi isim vererek sakladığı alan gibidir. Kutunun üzerinde isim ve içinde değer vardır.

```
int yas = 18;
decimal fiyat = 249.90m;
string isim = "Toygar";
bool aktif = true;
DateTime tarih = DateTime.Now;
```

int tam sayı. decimal para gibi hassas hesaplar için uygundur. string metin. bool true/false. DateTime tarih/saat.

4. Kullanıcıdan güvenli veri alma

```
Console.Write("Not: ");
string? giris = Console.ReadLine();

if (int.TryParse(giris, out int not))
{
    Console.WriteLine("Not: " + not);
}
else
{
    Console.WriteLine("Lütfen sayı gir.");
}
```

Önemli: Kullanıcıdan gelen veri her zaman beklediğin formatta olmayabilir. TryParse bunu güvenli şekilde kontrol etmeye yardım eder.

GÜN 8/48 — KONTROL AKIŞI VE METOTLAR

Hedef: Programın karar vermesini, tekrar etmesini ve kodu parçalara ayırmasını öğrenmek.

1. if / else = karar vermek

Günlük hayatta “eğer yağmur yağıyorsa şemsiye al, değilse alma” dersin. Programda da if/else ile buna benzer kararlar verirsın.

```
if (not >= 50)
{
    Console.WriteLine("Geçti");
}
else
{
    Console.WriteLine("Kaldı");
}
```

2. Döngüler = tekrar etmek

Bir işi 100 kez yapmak istediğinde aynı satırı 100 kez yazmak yerine döngü kullanırsın.

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}

foreach (string isim in isimler)
{
    Console.WriteLine(isim);
}
```

3. Metot = işi paketlemek

Bir metodu küçük bir makine gibi düşün. Girdi verirsın, bir iş yapar ve gerekiyorsa sonuç döndürür.

```
static int Topla(int a, int b)
{
    return a + b;
}

int sonuc = Topla(5, 3);
```

a ve b parametre. return sonucu dışarı verir. Topla(5, 3) metodu çağırır.

Üçlü ezber: if = karar • loop = tekrar • method = işi paketle.

GÜN 9/48 — KOLEKSİYONLAR VE LINQ

Hedef: Çok sayıda veriyi saklamak ve bu veriler üzerinde okunabilir sorgular yazmak.

1. Array

Array, birden fazla değeri aynı yapı içinde tutar. Boyutunun sabit olduğu senaryolar vardır.

```
string[] isimler = { "Ali", "Ayşe", "Mehmet" };
```

2. List

List, büyüyüp küçülebilen koleksiyon olarak sık kullanılır. kısmi listenin hangi türde veri tutacağını anlatır.

```
List<int> notlar = new List<int>();

notlar.Add(80);
notlar.Add(65);
notlar.Add(90);
```

3. Dictionary

Dictionary'yi sözlük gibi düşün: bir anahtar verirsin, karşılığındaki değeri bulursun.

```
Dictionary<int, string> ogrenciler = new();

ogrenciler.Add(1, "Ali");
ogrenciler.Add(2, "Ayşe");

Console.WriteLine(ogrenciler[1]);
```

4. LINQ nedir?

LINQ, koleksiyonlar üzerinde filtreleme, seçme, sıralama gibi işlemleri daha okunabilir şekilde yapmanı sağlayan sorgu yaklaşımıdır.

```
var basarililar = ogrencilerListesi
.Where(x => x.Not > 70)
.ToList();

var sirali = ogrencilerListesi
.OrderByDescending(x => x.Not)
.ToList();
```

Where filtreler. Select seçer/dönüştürür. OrderBy sıralar. Count sayar. FirstOrDefault ilk sonucu bulmaya çalışır.

Benzetme: LINQ'i büyük bir öğrenci listesinin içinden "70 üstündekileri getir ve nota göre sırala" diyen akıllı bir filtre gibi düşün.

BÖLÜM 4

OOP ile StajDesk'i Kurmak

GÜN 10/48 — SINIFLAR VE NESNELER

Hedef: Gerçek dünyadaki bir kavramı C# içinde class ve object olarak modellemek.

1. Önce gerçek hayat

Bir okulda “Öğrenci” diye bir kavram vardır. Her öğrencinin adı, numarası ve sınıfı olabilir. “Öğrenci” bir şablon gibi düşünülebilir; Ali ise o şablondan oluşturulmuş gerçek bir örnektir.

2. Class = şablon

```
class Oğrenci
{
    public string Ad { get; set; }
    public int Numara { get; set; }
}
```

3. Object = gerçek örnek

```
Oğrenci oğrenci1 = new Oğrenci();

oğrenci1.Ad = "Ali";
oğrenci1.Numara = 15;
```

Burada Oğrenci class, oğrenci1 ise o class'tan oluşturulan object'tir.

4. Property nedir?

Property, nesnenin taşıdığı bilginin kontrollü erişim noktasıdır. get okumayı, set değiştirmeyi temsil eder.

```
public string Title { get; set; }

ticket.Title = "Login problemi"; // set
Console.WriteLine(ticket.Title); // get
```

5. StajDesk Ticket class'ı

```
public class Ticket
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public string Status { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

6. Constructor = başlangıç kontrolü

Constructor, nesne oluşturulduğu anda otomatik çalışan özel metottur. Class'ın adıyla aynı isimde olur.

```
public Ticket(string title)
{
    if (string.IsNullOrEmpty(title))
    {
        throw new ArgumentException("Başlık boş olamaz.");
    }

    Title = title;
    Status = "Açık";
    CreatedAt = DateTime.Now;
}
```

Satırları oku:

- **title geldi mi? → kontrol et.**
- **boşsa → hata fırlat.**
- **doluysa → Title'a koy.**
- **yeni Ticket → Açık başlat.**
- **oluşturulma zamanını → şimdi yap.**

GÜN 10 — DEVAM: KAPSÜLLEME VE TİPLER

1. public ve private

public, dışarıdan erişilebilen üyeleri; private, sadece class'ın kendi içinde kullanılabilen üyeleri ifade eder.

```
public string Title { get; set; }

private string gizliBilgi;
```

2. Encapsulation / Kapsülleme

Kapsüllemeyi bir bankanın kasası gibi düşün. Kasayı herkesin açmasına izin vermezsin. Para yatırma/çekme işlemleri belirli kurallardan geçer. Yazılımda da veriyi kontrollü şekilde değiştirmek istersin.

```
private decimal bakiye;

public void ParaYatir(decimal miktar)
{
    if (miktar > 0)
    {
        bakiye += miktar;
    }
}
```

Burada dışarıdan doğrudan bakiye değiştirilmiyor. Sadece kuralı sağlayan bir metot üzerinden değişiyor.

3. Value Type ve Reference Type

Değer tipinde değişkenin değeri kopyalanır. Class gibi referans tiplerinde değişkenler aynı nesneyi gösterebilir.

```
int a = 10;
int b = a;
b = 20;
// a = 10, b = 20
```

```
Ticket ticket1 = new Ticket("Login");
Ticket ticket2 = ticket1;

ticket2.Title = "Şifre problemi";
// ticket1.Title da "Şifre problemi" olur.
```

Benzetme: Value type = fotokopisini aldın. Reference type = aynı dosyaya iki farklı kısayol verdin.

GÜN 11/48 — KALITIM, INTERFACE, ENUM

Hedef: OOP'nin sınıflar arası ilişkileri ve sözleşme mantığını anlamak.

1. Kalıtım / Inheritance

Bir class başka bir class'ın ortak özelliklerini devralabilir. Örneğin Admin bir Kullanıcı'dır; bu yüzden Kullanıcı'daki ortak özelliklerden yararlanabilir.

```
class Kullanici
{
    public string Ad { get; set; }
}

class Admin : Kullanici
{
    public bool Yetkili { get; set; }
}
```

2. Interface nedir?

Interface'i bir iş sözleşmesi gibi düşün. Sözleşme “şu işleri yapabilmelisin” der; işi nasıl yaptığını implement eden class belirler.

```
public interface ITicketRepository
{
    Task<Ticket> GetByIdAsync(int id);
    Task<List<Ticket>> GetAllAsync();
    Task AddAsync(Ticket ticket);
}
```

Bu interface'i uygulayan repository, bu metotların gerçek kodunu yazmalıdır.

3. Enum

Enum, sınırlı seçenekleri isimli hale getirir. Ticket durumu için “Open, InProgress, Resolved, Closed” gibi seçenekleri düzenli biçimde tutabilirsin.

```
public enum TicketStatus
{
    Open,
    InProgress,
    Resolved,
    Closed
}
```

4. Hata yönetimi

```
try
{
// hata çıkabilecek kod
}
catch (Exception ex)
{
Console.WriteLine(ex.Message);
}
finally
{
// gerekli temizlik
}
```

Günün cümlesi: Interface ne yapılacağını söyler; class nasıl yapılacağını gerçekleştirir.

GÜN 11 — DEVAM: REPOSITORY

1. Repository neden var?

Uygulamanın her yerinde “listeye ekle, listeden bul, sil” kodları yazmak yerine veri erişimini ayrı bir sınıfta toplamak daha düzenli bir yapı oluşturur.

```
public class InMemoryTicketRepository : ITicketRepository
{
    private readonly List<Ticket> tickets = new();

    public Task AddAsync(Ticket ticket)
    {
        tickets.Add(ticket);
        return Task.CompletedTask;
    }

    public Task<List<Ticket>> GetAllAsync()
    {
        return Task.FromResult(tickets);
    }
}
```

Burada InMemory verinin şimdilik bellekte/listede tutulduğunu anlatır. Daha sonra aynı interface'i kullanan PostgreSQL repository yazabilirsin.

2. Olmayan Id

```
var ticket = tickets.FirstOrDefault(x => x.Id == id);

if (ticket == null)
{
    throw new KeyNotFoundException(
        "Ticket bulunamadı.");
}
```

Büyük fikir: Interface sayesinde uygulamanın geri kalanı “veri nereden geliyor?” sorusuna fazla bağlanmaz. Repository bu ayrıntıyı üstlenir.

GÜN 12/48 — ASYNC / AWAIT

Hedef: Senkron ve asenkron çalışmanın farkını anlamak ve StajDesk v0 konsol yapısını tamamlamak.

1. Senkron nedir?

Bir görev bitmeden sıradaki göreve geçmediğini düşün. Garson mutfağın önünde bekliyor; sipariş gelmeden başka iş yapmıyor. Bu basit bir senkron düşüncedir.

2. Asenkron nedir?

Özellikle veritabanı veya ağ gibi bekleme içeren işlemlerde, beklerken başka işlerin ilerleyebilmesine izin veren yaklaşımlar kullanılabilir.

3. Task, async, await

```
public async Task<List<Ticket>> GetAllAsync()
{
    var tickets = await GetTicketsFromDatabaseAsync();
    return tickets;
}
```

Task devam eden/tamamlanacak işi temsil eder. async metodun asenkron işlemler kullanabileceğini belirtir. await sonucu beklerken uygun şekilde bekleme sağlar.

4. Neden web uygulamasında önemli?

Bir web sunucusunun veritabanından cevap beklediğini düşün. Her isteği gereksiz şekilde bloklamak yerine asenkron yaklaşım, bekleme sürelerinin daha verimli ele alınmasına yardımcı olabilir.

Önemli: async/await “her şeyi hızlandıran sihirli bir komut” değildir. Özellikle beklemeli I/O işlemlerinde kaynakların daha verimli kullanılmasına yardım eder.

GÜN 12 — STAJDESK v0'I BİRLEŞTİR

1. Konsol menüsü

Artık öğrendiğin parçaları tek uygulamada birleştiriyorsun. Kullanıcı menüden işlem seçer; program ilgili metodu çağırır.

```
=====
STAJDESK v0
=====

1 - Talep ekle
2 - Talepleri listele
3 - Durum güncelle
4 - Talep sil
0 - Çıkış

Seçiminiz:
```

2. Uygulamanın zihinsel modeli

```
KULLANICI
|
▼
KONSOL MENÜSÜ
|
▼
TICKET NESNESİ
|
▼
ITicketRepository
|
▼
InMemoryTicketRepository
|
▼
LIST<TICKET>
```

Bu yapıda her parçanın bir görevi vardır. Menü kullanıcıyla konuşur. Ticket veriyi temsil eder. Interface sözleşmeyi belirler. Repository veri erişimini yönetir.

3. Gün 12 sonunda ne biliyor olmalısın?

- Yeni bir Ticket oluşturabilmek.
- Başlığın boş olmasını engellemek.
- Ticket durumunu enum ile temsil edebilmek.
- Repository interface'i tanımlayabilmek.
- InMemory repository ile liste üzerinde işlem yapabilmek.
- async/await'in neden kullanıldığını temel seviyede açıklayabilmek.

STAJDESK v0: Artık sadece tek tek C# kodları yazmıyorsun. Öğrendiğin parçaları küçük bir uygulama mimarisi içinde birleştiriyorsun.

BÖLÜM 5

Kendini Test Et — Cevabı Önce Sen Ver

Aşağıdaki soruları PDF'ye bakmadan cevaplamaya çalış. Takıldığın sorunun ilgili Gün'üne geri dön.

- 1. Terminal nedir? cd ve dir ne işe yarar?**
- 2. Commit ile push arasındaki fark nedir?**
- 3. Branch neden kullanılır?**
- 4. Tarayıcı ile sunucu arasındaki iletişime ne denir?**
- 5. GET ile POST arasındaki fark nedir?**

- 6. 404 ile 500 arasındaki fark nedir?**
- 7. Docker image ile container arasındaki fark nedir?**
- 8. Dockerfile ne işe yarar?**
- 9. Volume neden kullanılır?**
- 10. Docker Compose neden kullanılır?**

- 11. int, decimal, string ve bool ne tutar?**
- 12. if/else ne yapar?**
- 13. for ve foreach ne zaman kullanılır?**
- 14. Metot nedir? Parametre ve return ne demektir?**
- 15. Array ile List arasındaki temel fark nedir?**

- 16. LINQ'te Where ne yapar?**
- 17. Class ile object arasındaki fark nedir?**
- 18. Property'de get ve set ne anlama gelir?**
- 19. Constructor ne zaman çalışır?**
- 20. Encapsulation neden önemlidir?**

21. Value type ile reference type arasındaki temel fark nedir?
22. Interface'i günlük hayattan bir sözleşmeyle nasıl açıklarsın?
23. Enum neden kullanılır?
24. Repository katmanı ne işe yarar?
25. async/await neden özellikle beklemeli işlemlerde kullanışlıdır?

SON BÖLÜM

Gün 1-12'nin Büyük Resmi

Şunu artık bir hikâye olarak anlatabilmelisin:

“Önce geliştirme ortamımı kurdum. Git ile kod geçmişini yönetmeyi öğrendim. Web'de client-server ve HTTP mantığını gördüm. Docker ile uygulama ortamını standartlaştırdım. C# ile temel programlama öğrendim. Koleksiyonları LINQ ile sorguladım. OOP ile Ticket'ı class olarak modelledim. Interface ve repository ile veri erişimini ayırdım. Sonra async/await ile beklemeli işlemlerin mantığını öğrendim.”

EZBERLEMEK YERİNE ŞU ZİNCİRİ HATIRLA

TERMINAL
↓
GIT
↓
WEB / HTTP
↓
DOCKER
↓
C#
↓
KOLEKSİYON + LINQ
↓
CLASS + OBJECT
↓
INTERFACE
↓
REPOSITORY
↓
ASYNC / AWAIT
↓
STAJDESK v0

Son kontrol

Bu zincirin her adımını 1-2 cümleyle açıklayabiliyorsan Gün 1-12'nin temelini gerçekten anlamışsın demektir.

MENTOR'A ANLATMA İPUCU

Bir kavramı anlatırken şu sırayı kullan: “Nedir?” → “Neden kullanılır?” → “Basit örnek nedir?” → “StajDesk'te nerede kullanıyorum?”